BEDROCK ARCHITECTURE

Bedrock C Far-Pointer Extensions

Source-Language Types and Semantics

Draft

CONTENTS

LIST OF TABLES

LIST OF FIGURES

1 SCOPE AND STATUS

This document defines the Bedrock-specific C source-language extensions for far data pointers, far function pointers, and cross-segment aliasing. It specifies declarator syntax, type compatibility, conversions, pointer operations, constant construction, compiler memory effects, and diagnostics.

This is not an ABI layer. Object representation, argument and return assignment, segment-register preservation, far-call frames, veneers, unwind, and dynamic linking are defined by the Bedrock C ABI and Bedrock ELF ABI. Compiler builtin and header availability is defined by the Bedrock Target Intrinsics document.

Extension name: `bedrock-c-far`

Language scope: ISO C with Bedrock target extensions

Not specified: C++ type system, name mangling, or C++ exceptions

2 TERMINOLOGY

far data pointer A C pointer kind carrying a pre-segment virtual address and the segment metadata whose ABI representation is defined by the Bedrock C ABI.

far function pointer A C function-pointer kind that selects a far call and return convention defined by the Bedrock C ABI.

canonical address The pre-segment virtual address that establishes C pointer identity independently of bounds metadata.

cross-segment alias barrier A compiler memory effect that reconciles ordinary near and segmented alias domains. It is not a hardware fence.

3 FAR POINTER TYPE SYSTEM

3.1 Declarator Binding

The implementation provides the pointer-kind qualifier `__far`, mapped to compiler address space 1. It applies to the pointer introduced by the immediately preceding declarator star, not to the pointed-to type.

```
T * __far p;               far pointer to T
const T * __far p;         far pointer to const T
T * const __far p;         const far pointer to T
T (* __far p)[N];          far pointer to an array
T * __far a[N];            array of far pointers
R (* __far fn)(Args...);   far function pointer
R __far function(Args...); far-native function
```

The qualifier is valid on every object- or function-pointer kind and composes with `const`, `volatile`, and `restrict` using their ordinary declarator meanings. Applying it to a pointed-to scalar, array element type, or nonpointer object type is a constraint violation.

Typedefs, `_Generic`, `typeof`, and equivalent type queries preserve the far pointer kind. `sizeof` and `_Alignof` report the size and alignment defined by the C ABI. An array parameter whose elements are far pointers adjusts to an ordinary pointer to a far-pointer element; `__far` does not migrate to the adjusted outer pointer.

A far pointer type cannot be the type of an atomic object or an operand of a compiler atomic builtin. Access through a far pointer to a pointed-to atomic object remains available when the C ABI supports the pointed-to atomic type.

3.2 Compatibility and Conversions

Far `void *` converts to and from far object pointers without changing the represented value. Far object pointers, far function pointers, ordinary object pointers, and ordinary function pointers remain distinct categories. There is no object/function pointer compatibility and no implicit near/far conversion.

Explicit near-to-far conversion retains the near pointer's canonical address and attaches the current canonical disabled or bounds-only data context. Explicit far-to-near conversion retains the canonical address but succeeds only when it is valid in the current ordinary data context. A null source converts to the ordinary null pointer. For a non-null source, evaluating the conversion when the canonical address is not admitted by the current ordinary data context has undefined behavior. No runtime check or trap is required at the conversion point, and the conversion does not probe page translation.

Every declaration and definition of a far-native function must agree on the far function kind. Taking its address produces the corresponding far function pointer. A mismatch between near and far declarations requires a diagnostic. In the current draft, every explicit conversion or cast between ordinary and far function-pointer types is a constraint violation that requires a diagnostic, including when the operand names a known function symbol. Such a cast never requests a veneer, and a conforming implementation need not synthesize one. An implementation may expose a separately named veneer extension for a known, prototyped, nonvariadic function; any veneer emitted by that extension follows the typed veneer ABI defined by the C ABI.

3.3 Pointer Operations and Identity

Pointer arithmetic changes the canonical address while preserving the associated bounds context. Pointer difference and relational comparison are defined only for pointers into the same C array object and use canonical addresses. Equality and hashing use canonical address identity: every zero-address value is null, and differing image words do not make null values or otherwise equal addresses unequal.

The builtin `__builtin_bedrock_far_same_encoding(left,right)` compares the complete ABI representation when raw encoding identity is required. A zero-address null value may therefore have distinct raw encodings when its image words differ. A value with invalid or noncanonical metadata may be copied, compared, or converted to its matching raw carrier without faulting, but dereference or call is invalid. The C ABI validates a scalar far-pointer argument or result, including

a null value's retained image, before control transfer. An aggregate crosses the boundary as an uninspected object representation, so a contained far-pointer encoding is instead validated when an operation extracts and uses it as a scalar far pointer. The C ABI defines the null identity and raw-carrier representations.

3.4 Pointer and Integer Conversion

Ordinary `uintptr_t` and `intptr_t` remain near-pointer carriers. Explicit conversion to the matching far raw carrier preserves the complete representation; conversion back restores both words. A zero address denotes null without clearing its restored image word. A matching-carrier round trip compares equal and retains its bounds context and raw encoding.

The builtin `__builtin_bedrock_far_address(pointer)` returns the canonical 64-bit pre-segment address without validation or faulting. Conversion through an ordinary 64-bit carrier is rejected unless an explicitly implementation-defined truncation operation is requested.

4 FAR POINTER CONSTRUCTION

The compiler-owned header `<bedrockintrin.h>` exposes typed macros over the matching target builtins. These operations construct values; they do not expose a C structure representation.

The included `<bedrockfarintrin.h>` also exposes `__BEDROCK_SEGMENT_IMAGE`, `__BEDROCK_SEGMENT_IMAGE_FOR_BASE`, and `__BEDROCK_SEGMENT_DISABLED`. These are pure 64-bit value expressions, not architectural segment-register operations. The first form packs the segment fields and normalizes the bounds-only operand to zero or one. The second accepts a virtual byte address, discards its low 12 bits, and encodes the containing 4096-byte page in the base-page field. It imposes no alignment precondition and does not preserve a misaligned operand's original byte address. The disabled spelling is the canonical all-zero image. The Bedrock Target Intrinsics document defines the complete signatures, field-width, alignment, and validation contract.

Each macro's first argument names a far-qualified object or function pointer type. The typed initializer accepts a pre-segment address expressed in the resulting bounds-only segment's coordinate space and changes a nonzero-mantissa image to bounds-only without adding its base. The flat form attaches the disabled image. The translated-source form adds base and offset, uses the low 64 bits of the sum as the constructed pointer's pre-segment address, sets bounds-only mode, and preserves the image's bounds. The null form produces the ABI all-zero null value. The translated-source form requires an enabled image with nonzero mantissa and bounds-only mode clear. An already-linear address paired with a bounds-only image uses the typed initializer instead.

Construction writes no segment register, probes no bounds, walks no page table, and performs no memory access. It neither detects nor reports base-plus-offset overflow. The converted bounds-only image is retained whether the low 64-bit modular sum is zero or nonzero. A zero sum denotes null without changing that image. For a valid source image, an in-range offset cannot overflow; a nonzero result from an out-of-range or wrapping offset lies outside the retained bounds and fails at the dependent access or control transfer. A one-past address may therefore be constructed without access. Image validation occurs when a scalar value crosses a C ABI argument or return boundary and before dereference or call; bounds checking occurs at the dependent access or control transfer.

When every value operand is an integer constant expression and the pointer type is complete enough for its cast, the macro result is accepted as a constant initializer for an object of the resulting far-pointer type. This target extension does not classify the pointer-valued result itself as an integer constant expression.

5 ACCESS AND CROSS-SEGMENT ALIASING

5.1 Access Semantics

Dereference uses the address and segment metadata carried by the far pointer. An incoming register argument uses its assigned `GS2` through `GS5` segment channel. A memory, local, or stack value is established in scratch `GS1`. The implementation must re-establish the selected caller-saved GS image after any operation that may clobber it and before a dependent access. Invalid metadata raises `INVALID_CONTROL_STATE`; a failed bound raises `PAGE_FAULT`. Ordinary volatile access-count and ordering requirements remain in force.

5.2 Alias Identity

Compiler regions with no cross-segment operation may partition near and segmented memory into distinct alias domains. A cross-segment operation reconciles the affected domains with a cross-segment alias barrier. Two accesses alias when their canonical pre-segment byte ranges overlap and ordinary C effective-type rules permit aliasing. Segment metadata may be unknown at compile time, and visible far operations require no source annotation.

Table 5-1. Cross-Segment Alias-Barrier Triggers

Trigger	Compiler memory effect
far data load	reconcile potentially aliased state before the read
far data store	reconcile before the write and invalidate affected cached values afterward
far data RMW	apply both the read and write effects
direct far call	materialize dirty escaped state before the call and invalidate cached escaped state afterward
indirect far call	same escaped-memory effect as a direct far call
hidden access	inline assembly, intrinsic, or external code declared with `cross_segment_access`

Pointer construction, copying, arithmetic, conversion, and comparison are not triggers. Before a triggering access, dirty potentially aliased escaped state must be materialized. After a write, RMW, or far call, potentially aliased cached values must be invalidated; lazy reload is permitted. Objects proven not to overlap need no spill or invalidation. Adjacent barriers may be coalesced only when observable C behavior is unchanged.

A function containing a trigger acquires the inferred `__attribute__((cross_segment_access))` property. Explicit use declares a hidden effect. The property is not part of the function type and does not change calling convention or symbol identity. Its memory summary survives inlining and LTO. Contradictory `readnone`, `const`, `pure`, or equivalent annotations are rejected.

Strict aliasing, `restrict`, volatile, atomic-object identity, and data races are evaluated using canonical address ranges. Different bounds contexts may designate the same atomic object. The alias barrier is not a hardware memory fence.

5.3 TLS and Lifetime Semantics

A C far pointer never carries TLS selector metadata. Taking the address of a TLS object first materializes the concrete pre-segment address of the calling thread's instance. Explicit conversion attaches ordinary far data metadata. Copying the result to another thread still designates the originating instance while it is alive; thread exit ends that lifetime. Loader unload likewise ends the lifetime of pointers to unloaded data and function entries.

5.4 Worked Access

```
uint64_t load_far_u64(const uint64_t * __far p) { return *p; }
```

The C ABI assigns the address to an incoming Rn location and the validated segment image to the matching `GS2` through `GS5` location. The compiler emits one dependent load through that segment register. The exact instruction sequence is an implementation choice as long as those observable rules hold.

6 FAR FUNCTION LANGUAGE SEMANTICS

A far function pointer declaration such as `int (* __far handler)(int)` selects the far function-pointer type. Calling through it uses the far-call convention defined by the C ABI. The declaration `int __far handler(int)` defines a far-native entry; direct calls to it use the same convention. Ordinary function declarations and pointers retain the near convention.

Far function pointers are incompatible with near function pointers and far data pointers. Native variadic far functions are supported, but automatic variadic or unprototyped cross-convention adaptation is forbidden; source code must provide an explicitly typed wrapper.

6.1 Worked Indirect Call

```
int invoke(int (* __far fn)(int), int value) { return fn(value); }
```

The compiler preserves the address and segment image of `fn` while assigning `value` by the ordinary integer argument rules, copies the image from its GS channel to the Rn operand required by `LCALL`, emits the C ABI far call, and expects the target to use the corresponding far return.